

3 | Linear Neural Networks

Before we get into the details of deep neural networks, we need to cover the basics of neural network training. In this chapter, we will cover the entire training process, including defining simple neural network architectures, handling data, specifying a loss function, and training the model. In order to make things easier to grasp, we begin with the simplest concepts. Fortunately, classic statistical learning techniques such as linear and softmax regression can be cast as *linear* neural networks. Starting from these classic algorithms, we will introduce you to the basics, providing the basis for more complex techniques in the rest of the book.

3.1 Linear Regression

Regression refers to a set of methods for modeling the relationship between one or more independent variables and a dependent variable. In the natural sciences and social sciences, the purpose of regression is most often to *characterize* the relationship between the inputs and outputs. Machine learning, on the other hand, is most often concerned with *prediction*.

Regression problems pop up whenever we want to predict a numerical value. Common examples include predicting prices (of homes, stocks, etc.), predicting length of stay (for patients in the hospital), demand forecasting (for retail sales), among countless others. Not every prediction problem is a classic regression problem. In subsequent sections, we will introduce classification problems, where the goal is to predict membership among a set of categories.

3.1.1 Basic Elements of Linear Regression

Linear regression may be both the simplest and most popular among the standard tools to regression. Dating back to the dawn of the 19th century, linear regression flows from a few simple assumptions. First, we assume that the relationship between the independent variables $\mathbf{x}$ and the dependent variable y is linear, i.e., that y can be expressed as a weighted sum of the elements in $\mathbf{x}$, given some noise on the observations. Second, we assume that any noise is well-behaved (following a Gaussian distribution).

To motivate the approach, let us start with a running example. Suppose that we wish to estimate the prices of houses (in dollars) based on their area (in square feet) and age (in years). To actually develop a model for predicting house prices, we would need to get our hands on a dataset consisting of sales for which we know the sale price, area, and age for each home. In the terminology of machine learning, the dataset is called a *training dataset* or *training set*, and each row (here the data corresponding to one sale) is called an *example* (or *data point*, *data instance*, *sample*). The thing we are trying to predict (price) is called a *label* (or *target*). The independent variables (age and area) upon which the predictions are based are called *features* (or *covariates*).

Typically, we will use n to denote the number of examples in our dataset. We index the data examples by i , denoting each input as $\mathbf{x}^{(i)} = [x_1^{(i)}, x_2^{(i)}]^\top$ and the corresponding label as $y^{(i)}$.

Linear Model

The linearity assumption just says that the target (price) can be expressed as a weighted sum of the features (area and age):

$$\text{price} = w_{\text{area}} \cdot \text{area} + w_{\text{age}} \cdot \text{age} + b. \quad (3.1.1)$$

In (3.1.1), w_{area} and w_{age} are called *weights*, and b is called a *bias* (also called an *offset* or *intercept*). The weights determine the influence of each feature on our prediction and the bias just says what value the predicted price should take when all of the features take value 0. Even if we will never see any homes with zero area, or that are precisely zero years old, we still need the bias or else we will limit the expressivity of our model. Strictly speaking, (3.1.1) is an *affine transformation* of input features, which is characterized by a *linear transformation* of features via weighted sum, combined with a *translation* via the added bias.

Given a dataset, our goal is to choose the weights $\mathbf{w}$ and the bias b such that on average, the predictions made according to our model best fit the true prices observed in the data. Models whose output prediction is determined by the affine transformation of input features are *linear models*, where the affine transformation is specified by the chosen weights and bias.

In disciplines where it is common to focus on datasets with just a few features, explicitly expressing models long-form like this is common. In machine learning, we usually work with high-dimensional datasets, so it is more convenient to employ linear algebra notation. When our inputs consist of d features, we express our prediction $\hat{y}$ (in general the “hat” symbol denotes estimates) as

$$\hat{y} = w_1 x_1 + \dots + w_d x_d + b. \quad (3.1.2)$$

Collecting all features into a vector $\mathbf{x} \in \mathbb{R}^d$ and all weights into a vector $\mathbf{w} \in \mathbb{R}^d$, we can express our model compactly using a dot product:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b. \quad (3.1.3)$$

In (3.1.3), the vector $\mathbf{x}$ corresponds to features of a single data example. We will often find it convenient to refer to features of our entire dataset of n examples via the *design matrix* $\mathbf{X} \in \mathbb{R}^{n \times d}$. Here, $\mathbf{X}$ contains one row for every example and one column for every feature.

For a collection of features $\mathbf{X}$, the predictions $\hat{\mathbf{y}} \in \mathbb{R}^n$ can be expressed via the matrix-vector product:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b, \quad (3.1.4)$$

where broadcasting (see Section 2.1.3) is applied during the summation. Given features of a training dataset $\mathbf{X}$ and corresponding (known) labels $\mathbf{y}$, the goal of linear regression is to find the weight vector $\mathbf{w}$ and the bias term b that given features of a new data example sampled from the same distribution as $\mathbf{X}$, the new example’s label will (in expectation) be predicted with the lowest error.

Even if we believe that the best model for predicting y given $\mathbf{x}$ is linear, we would not expect to find a real-world dataset of n examples where $y^{(i)}$ exactly equals $\mathbf{w}^\top \mathbf{x}^{(i)} + b$ for all $1 \leq i \leq n$. For example, whatever instruments we use to observe the features $\mathbf{X}$ and labels $\mathbf{y}$ might suffer small

amount of measurement error. Thus, even when we are confident that the underlying relationship is linear, we will incorporate a noise term to account for such errors.

Before we can go about searching for the best *parameters* (or *model parameters*) $\mathbf{w}$ and b , we will need two more things: (i) a quality measure for some given model; and (ii) a procedure for updating the model to improve its quality.

Loss Function

Before we start thinking about how to *fit* data with our model, we need to determine a measure of *fitness*. The *loss function* quantifies the distance between the *real* and *predicted* value of the target. The loss will usually be a non-negative number where smaller values are better and perfect predictions incur a loss of 0. The most popular loss function in regression problems is the squared error. When our prediction for an example i is $\hat{y}^{(i)}$ and the corresponding true label is $y^{(i)}$, the squared error is given by:

$$l^{(i)}(\mathbf{w}, b) = \frac{1}{2} \left(\hat{y}^{(i)} - y^{(i)} \right)^2. \quad (3.1.5)$$

The constant $\frac{1}{2}$ makes no real difference but will prove notationally convenient, canceling out when we take the derivative of the loss. Since the training dataset is given to us, and thus out of our control, the empirical error is only a function of the model parameters. To make things more concrete, consider the example below where we plot a regression problem for a one-dimensional case as shown in Fig. 3.1.1.

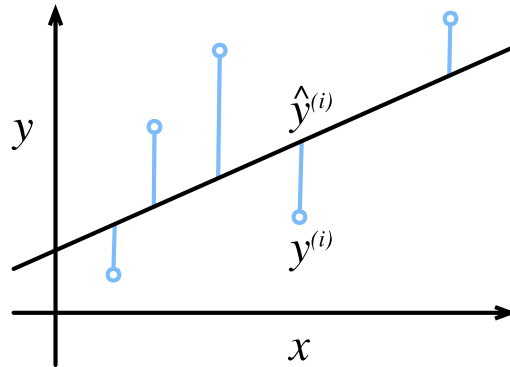


Fig. 3.1.1: Fit data with a linear model.

Note that large differences between estimates $\hat{y}^{(i)}$ and observations $y^{(i)}$ lead to even larger contributions to the loss, due to the quadratic dependence. To measure the quality of a model on the entire dataset of n examples, we simply average (or equivalently, sum) the losses on the training set.

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n l^{(i)}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)} \right)^2. \quad (3.1.6)$$

When training the model, we want to find parameters $(\mathbf{w}^*, b^*)$ that minimize the total loss across all training examples:

$$\mathbf{w}^*, b^* = \underset{\mathbf{w}, b}{\operatorname{argmin}} L(\mathbf{w}, b). \quad (3.1.7)$$

Analytic Solution

Linear regression happens to be an unusually simple optimization problem. Unlike most other models that we will encounter in this book, linear regression can be solved analytically by applying a simple formula. To start, we can subsume the bias b into the parameter $\mathbf{w}$ by appending a column to the design matrix consisting of all ones. Then our prediction problem is to minimize $\|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$. There is just one critical point on the loss surface and it corresponds to the minimum of the loss over the entire domain. Taking the derivative of the loss with respect to $\mathbf{w}$ and setting it equal to zero yields the analytic (closed-form) solution:

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (3.1.8)$$

While simple problems like linear regression may admit analytic solutions, you should not get used to such good fortune. Although analytic solutions allow for nice mathematical analysis, the requirement of an analytic solution is so restrictive that it would exclude all of deep learning.

Minibatch Stochastic Gradient Descent

Even in cases where we cannot solve the models analytically, it turns out that we can still train models effectively in practice. Moreover, for many tasks, those difficult-to-optimize models turn out to be so much better that figuring out how to train them ends up being well worth the trouble.

The key technique for optimizing nearly any deep learning model, and which we will call upon throughout this book, consists of iteratively reducing the error by updating the parameters in the direction that incrementally lowers the loss function. This algorithm is called *gradient descent*.

The most naive application of gradient descent consists of taking the derivative of the loss function, which is an average of the losses computed on every single example in the dataset. In practice, this can be extremely slow: we must pass over the entire dataset before making a single update. Thus, we will often settle for sampling a random minibatch of examples every time we need to compute the update, a variant called *minibatch stochastic gradient descent*.

In each iteration, we first randomly sample a minibatch $\mathcal{B}$ consisting of a fixed number of training examples. We then compute the derivative (gradient) of the average loss on the minibatch with regard to the model parameters. Finally, we multiply the gradient by a predetermined positive value η and subtract the resulting term from the current parameter values.

We can express the update mathematically as follows (∂ denotes the partial derivative):

$$(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \partial_{(\mathbf{w}, b)} l^{(i)}(\mathbf{w}, b). \quad (3.1.9)$$

To summarize, steps of the algorithm are the following: (i) we initialize the values of the model parameters, typically at random; (ii) we iteratively sample random minibatches from the data, updating the parameters in the direction of the negative gradient. For quadratic losses and affine transformations, we can write this out explicitly as follows:

$$\begin{aligned} \mathbf{w} &\leftarrow \mathbf{w} - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \partial_{\mathbf{w}} l^{(i)}(\mathbf{w}, b) = \mathbf{w} - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \mathbf{x}^{(i)} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)} \right), \\ b &\leftarrow b - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \partial_b l^{(i)}(\mathbf{w}, b) = b - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)} \right). \end{aligned} \quad (3.1.10)$$

Note that $\mathbf{w}$ and $\mathbf{x}$ are vectors in (3.1.10). Here, the more elegant vector notation makes the math much more readable than expressing things in terms of coefficients, say $w_1, w_2, \dots, w_d$. The set

cardinality $|\mathcal{B}|$ represents the number of examples in each minibatch (the *batch size*) and η denotes the *learning rate*. We emphasize that the values of the batch size and learning rate are manually pre-specified and not typically learned through model training. These parameters that are tunable but not updated in the training loop are called *hyperparameters*. *Hyperparameter tuning* is the process by which hyperparameters are chosen, and typically requires that we adjust them based on the results of the training loop as assessed on a separate *validation dataset* (or *validation set*).

After training for some predetermined number of iterations (or until some other stopping criteria are met), we record the estimated model parameters, denoted $\hat{\mathbf{w}}, \hat{b}$. Note that even if our function is truly linear and noiseless, these parameters will not be the exact minimizers of the loss because, although the algorithm converges slowly towards the minimizers it cannot achieve it exactly in a finite number of steps.

Linear regression happens to be a learning problem where there is only one minimum over the entire domain. However, for more complicated models, like deep networks, the loss surfaces contain many minima. Fortunately, for reasons that are not yet fully understood, deep learning practitioners seldom struggle to find parameters that minimize the loss *on training sets*. The more formidable task is to find parameters that will achieve low loss on data that we have not seen before, a challenge called *generalization*. We return to these topics throughout the book.

Making Predictions with the Learned Model

Given the learned linear regression model $\hat{\mathbf{w}}^\top \mathbf{x} + \hat{b}$, we can now estimate the price of a new house (not contained in the training data) given its area x_1 and age x_2 . Estimating targets given features is commonly called *prediction* or *inference*.

We will try to stick with *prediction* because calling this step *inference*, despite emerging as standard jargon in deep learning, is somewhat of a misnomer. In statistics, *inference* more often denotes estimating parameters based on a dataset. This misuse of terminology is a common source of confusion when deep learning practitioners talk to statisticians.

3.1.2 Vectorization for Speed

When training our models, we typically want to process whole minibatches of examples simultaneously. Doing this efficiently requires that we vectorize the calculations and leverage fast linear algebra libraries rather than writing costly for-loops in Python.

```
%matplotlib inline
import math
import time
import numpy as np
import torch
from d2l import torch as d2l
```

To illustrate why this matters so much, we can consider two methods for adding vectors. To start we instantiate two 10000-dimensional vectors containing all ones. In one method we will loop over the vectors with a Python for-loop. In the other method we will rely on a single call to `+`.

```
n = 10000
a = torch.ones(n)
b = torch.ones(n)
```

Since we will benchmark the running time frequently in this book, let us define a timer.

```
class Timer: #@save
    """Record multiple running times."""
    def __init__(self):
        self.times = []
        self.start()

    def start(self):
        """Start the timer."""
        self.tik = time.time()

    def stop(self):
        """Stop the timer and record the time in a list."""
        self.times.append(time.time() - self.tik)
        return self.times[-1]

    def avg(self):
        """Return the average time."""
        return sum(self.times) / len(self.times)

    def sum(self):
        """Return the sum of time."""
        return sum(self.times)

    def cumsum(self):
        """Return the accumulated time."""
        return np.array(self.times).cumsum().tolist()
```

Now we can benchmark the workloads. First, we add them, one coordinate at a time, using a for-loop.

```
c = torch.zeros(n)
timer = Timer()
for i in range(n):
    c[i] = a[i] + b[i]
f'{timer.stop():.5f} sec'
```

```
'0.10353 sec'
```

Alternatively, we rely on the reloaded + operator to compute the elementwise sum.

```
timer.start()
d = a + b
f'{timer.stop():.5f} sec'
```

```
'0.00028 sec'
```

You probably noticed that the second method is dramatically faster than the first. Vectorizing code often yields order-of-magnitude speedups. Moreover, we push more of the mathematics to the library and need not write as many calculations ourselves, reducing the potential for errors.

3.1.3 The Normal Distribution and Squared Loss

While you can already get your hands dirty using only the information above, in the following we can more formally motivate the squared loss objective via assumptions about the distribution of noise.

Linear regression was invented by Gauss in 1795, who also discovered the normal distribution (also called the *Gaussian*). It turns out that the connection between the normal distribution and linear regression runs deeper than common parentage. To refresh your memory, the probability density of a normal distribution with mean μ and variance σ^2 (standard deviation σ) is given as

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(x - \mu)^2\right). \quad (3.1.11)$$

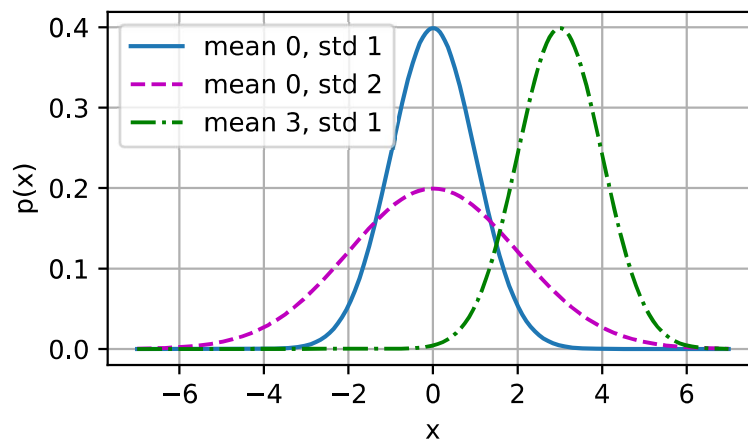
Below we define a Python function to compute the normal distribution.

```
def normal(x, mu, sigma):
    p = 1 / math.sqrt(2 * math.pi * sigma**2)
    return p * np.exp(-0.5 / sigma**2 * (x - mu)**2)
```

We can now visualize the normal distributions.

```
# Use numpy again for visualization
x = np.arange(-7, 7, 0.01)

# Mean and standard deviation pairs
params = [(0, 1), (0, 2), (3, 1)]
d2l.plot(x, [normal(x, mu, sigma) for mu, sigma in params], xlabel='x',
        ylabel='p(x)', figsize=(4.5, 2.5),
        legend=[f'mean {mu}, std {sigma}' for mu, sigma in params])
```



As we can see, changing the mean corresponds to a shift along the x -axis, and increasing the variance spreads the distribution out, lowering its peak.

One way to motivate linear regression with the mean squared error loss function (or simply squared loss) is to formally assume that observations arise from noisy observations, where the noise is normally distributed as follows:

$$y = \mathbf{w}^\top \mathbf{x} + b + \epsilon \text{ where } \epsilon \sim \mathcal{N}(0, \sigma^2). \quad (3.1.12)$$

Thus, we can now write out the *likelihood* of seeing a particular y for a given $\mathbf{x}$ via

$$P(y | \mathbf{x}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(y - \mathbf{w}^\top \mathbf{x} - b)^2\right). \quad (3.1.13)$$

Now, according to the principle of maximum likelihood, the best values of parameters $\mathbf{w}$ and b are those that maximize the *likelihood* of the entire dataset:

$$P(\mathbf{y} | \mathbf{X}) = \prod_{i=1}^n p(y^{(i)} | \mathbf{x}^{(i)}). \quad (3.1.14)$$

Estimators chosen according to the principle of maximum likelihood are called *maximum likelihood estimators*. While, maximizing the product of many exponential functions, might look difficult, we can simplify things significantly, without changing the objective, by maximizing the log of the likelihood instead. For historical reasons, optimizations are more often expressed as minimization rather than maximization. So, without changing anything we can minimize the *negative log-likelihood* $-\log P(\mathbf{y} | \mathbf{X})$. Working out the mathematics gives us:

$$-\log P(\mathbf{y} | \mathbf{X}) = \sum_{i=1}^n \frac{1}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \left(y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)} - b\right)^2. \quad (3.1.15)$$

Now we just need one more assumption that σ is some fixed constant. Thus we can ignore the first term because it does not depend on $\mathbf{w}$ or b . Now the second term is identical to the squared error loss introduced earlier, except for the multiplicative constant $\frac{1}{\sigma^2}$. Fortunately, the solution does not depend on σ . It follows that minimizing the mean squared error is equivalent to maximum likelihood estimation of a linear model under the assumption of additive Gaussian noise.

3.1.4 From Linear Regression to Deep Networks

So far we only talked about linear models. While neural networks cover a much richer family of models, we can begin thinking of the linear model as a neural network by expressing it in the language of neural networks. To begin, let us start by rewriting things in a “layer” notation.

Neural Network Diagram

Deep learning practitioners like to draw diagrams to visualize what is happening in their models. In Fig. 3.1.2, we depict our linear regression model as a neural network. Note that these diagrams highlight the connectivity pattern such as how each input is connected to the output, but not the values taken by the weights or biases.

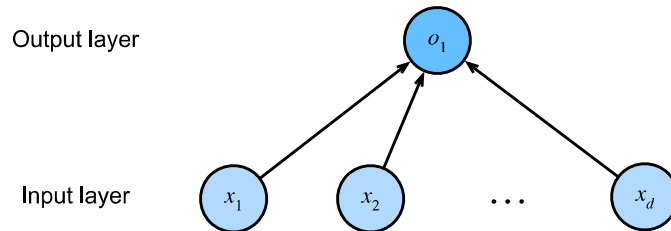


Fig. 3.1.2: Linear regression is a single-layer neural network.

For the neural network shown in Fig. 3.1.2, the inputs are $x_1, \dots, x_d$, so the *number of inputs* (or *feature dimensionality*) in the input layer is d . The output of the network in Fig. 3.1.2 is o_1 , so the

number of outputs in the output layer is 1. Note that the input values are all *given* and there is just a single *computed* neuron. Focusing on where computation takes place, conventionally we do not consider the input layer when counting layers. That is to say, the *number of layers* for the neural network in Fig. 3.1.2 is 1. We can think of linear regression models as neural networks consisting of just a single artificial neuron, or as single-layer neural networks.

Since for linear regression, every input is connected to every output (in this case there is only one output), we can regard this transformation (the output layer in Fig. 3.1.2) as a *fully-connected layer* or *dense layer*. We will talk a lot more about networks composed of such layers in the next chapter.

Biology

Since linear regression (invented in 1795) predates computational neuroscience, it might seem anachronistic to describe linear regression as a neural network. To see why linear models were a natural place to begin when the cyberneticists/neurophysiologists Warren McCulloch and Walter Pitts began to develop models of artificial neurons, consider the cartoonish picture of a biological neuron in Fig. 3.1.3, consisting of *dendrites* (input terminals), the *nucleus* (CPU), the *axon* (output wire), and the *axon terminals* (output terminals), enabling connections to other neurons via *synapses*.

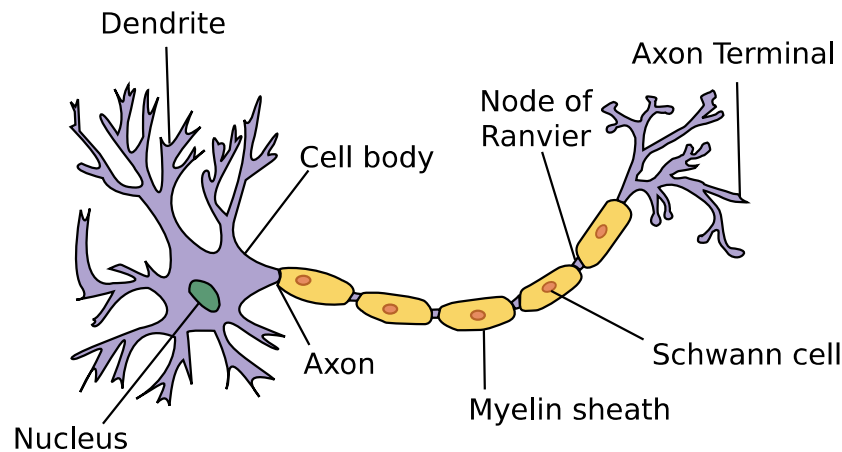


Fig. 3.1.3: The real neuron.

Information x_i arriving from other neurons (or environmental sensors such as the retina) is received in the dendrites. In particular, that information is weighted by *synaptic weights* w_i determining the effect of the inputs (e.g., activation or inhibition via the product $x_i w_i$). The weighted inputs arriving from multiple sources are aggregated in the nucleus as a weighted sum $y = \sum_i x_i w_i + b$, and this information is then sent for further processing in the axon y , typically after some nonlinear processing via $\sigma(y)$. From there it either reaches its destination (e.g., a muscle) or is fed into another neuron via its dendrites.

Certainly, the high-level idea that many such units could be cobbled together with the right connectivity and right learning algorithm, to produce far more interesting and complex behavior than any one neuron alone could express owes to our study of real biological neural systems.

At the same time, most research in deep learning today draws little direct inspiration in neuroscience. We invoke Stuart Russell and Peter Norvig who, in their classic AI text book *Artificial Intelligence: A Modern Approach* (Russell & Norvig, 2016), pointed out that although airplanes might have been *inspired* by birds, ornithology has not been the primary driver of aeronautics innovation

for some centuries. Likewise, inspiration in deep learning these days comes in equal or greater measure from mathematics, statistics, and computer science.

Summary

- Key ingredients in a machine learning model are training data, a loss function, an optimization algorithm, and quite obviously, the model itself.
- Vectorizing makes everything better (mostly math) and faster (mostly code).
- Minimizing an objective function and performing maximum likelihood estimation can mean the same thing.
- Linear regression models are neural networks, too.

Exercises

1. Assume that we have some data $x_1, \dots, x_n \in \mathbb{R}$. Our goal is to find a constant b such that $\sum_i (x_i - b)^2$ is minimized.
 1. Find an analytic solution for the optimal value of b .
 2. How does this problem and its solution relate to the normal distribution?
2. Derive the analytic solution to the optimization problem for linear regression with squared error. To keep things simple, you can omit the bias b from the problem (we can do this in principled fashion by adding one column to $\mathbf{X}$ consisting of all ones).
 1. Write out the optimization problem in matrix and vector notation (treat all the data as a single matrix, and all the target values as a single vector).
 2. Compute the gradient of the loss with respect to w .
 3. Find the analytic solution by setting the gradient equal to zero and solving the matrix equation.
 4. When might this be better than using stochastic gradient descent? When might this method break?
3. Assume that the noise model governing the additive noise ϵ is the exponential distribution. That is, $p(\epsilon) = \frac{1}{2} \exp(-|\epsilon|)$.
 1. Write out the negative log-likelihood of the data under the model $-\log P(\mathbf{y} | \mathbf{X})$.
 2. Can you find a closed form solution?
 3. Suggest a stochastic gradient descent algorithm to solve this problem. What could possibly go wrong (hint: what happens near the stationary point as we keep on updating the parameters)? Can you fix this?

Discussions⁴⁸

⁴⁸ <https://discuss.d2l.ai/t/258>

3.2 Linear Regression Implementation from Scratch

Now that you understand the key ideas behind linear regression, we can begin to work through a hands-on implementation in code. In this section, we will implement the entire method from scratch, including the data pipeline, the model, the loss function, and the minibatch stochastic gradient descent optimizer. While modern deep learning frameworks can automate nearly all of this work, implementing things from scratch is the only way to make sure that you really know what you are doing. Moreover, when it comes time to customize models, defining our own layers or loss functions, understanding how things work under the hood will prove handy. In this section, we will rely only on tensors and auto differentiation. Afterwards, we will introduce a more concise implementation, taking advantage of bells and whistles of deep learning frameworks.

```
%matplotlib inline
import random
import torch
from d2l import torch as d2l
```

3.2.1 Generating the Dataset

To keep things simple, we will construct an artificial dataset according to a linear model with additive noise. Our task will be to recover this model's parameters using the finite set of examples contained in our dataset. We will keep the data low-dimensional so we can visualize it easily. In the following code snippet, we generate a dataset containing 1000 examples, each consisting of 2 features sampled from a standard normal distribution. Thus our synthetic dataset will be a matrix $\mathbf{X} \in \mathbb{R}^{1000 \times 2}$.

The true parameters generating our dataset will be $\mathbf{w} = [2, -3.4]^\top$ and $b = 4.2$, and our synthetic labels will be assigned according to the following linear model with the noise term ϵ :

$$\mathbf{y} = \mathbf{X}\mathbf{w} + b + \epsilon. \quad (3.2.1)$$

You could think of ϵ as capturing potential measurement errors on the features and labels. We will assume that the standard assumptions hold and thus that ϵ obeys a normal distribution with mean of 0. To make our problem easy, we will set its standard deviation to 0.01. The following code generates our synthetic dataset.

```
def synthetic_data(w, b, num_examples):  #@save
    """Generate  $y = Xw + b + \text{noise}$ ."""
    X = torch.normal(0, 1, (num_examples, len(w)))
    y = torch.matmul(X, w) + b
    y += torch.normal(0, 0.01, y.shape)
    return X, y.reshape((-1, 1))

true_w = torch.tensor([2, -3.4])
true_b = 4.2
features, labels = synthetic_data(true_w, true_b, 1000)
```

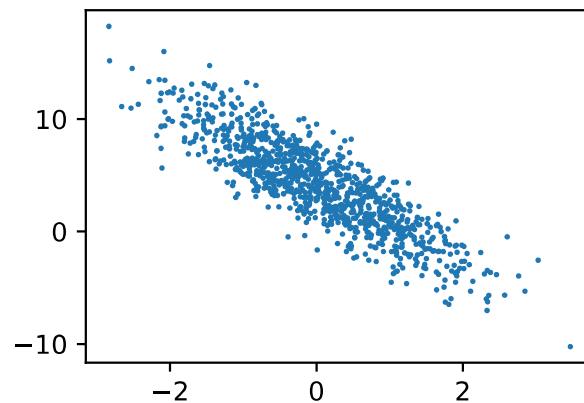
Note that each row in features consists of a 2-dimensional data example and that each row in labels consists of a 1-dimensional label value (a scalar).

```
print('features:', features[0], '\nlabel:', labels[0])
```

```
features: tensor([-1.4598, -0.7918])
label: tensor([3.9780])
```

By generating a scatter plot using the second feature `features[:, 1]` and `labels`, we can clearly observe the linear correlation between the two.

```
d2l.set_figsize()
# The semicolon is for displaying the plot only
d2l.plt.scatter(features[:, (1)].detach().numpy(), labels.detach().numpy(), 1);
```



3.2.2 Reading the Dataset

Recall that training models consists of making multiple passes over the dataset, grabbing one minibatch of examples at a time, and using them to update our model. Since this process is so fundamental to training machine learning algorithms, it is worth defining a utility function to shuffle the dataset and access it in minibatches.

In the following code, we define the `data_iter` function to demonstrate one possible implementation of this functionality. The function takes a batch size, a matrix of features, and a vector of labels, yielding minibatches of the size `batch_size`. Each minibatch consists of a tuple of features and labels.

```
def data_iter(batch_size, features, labels):
    num_examples = len(features)
    indices = list(range(num_examples))
    # The examples are read at random, in no particular order
    random.shuffle(indices)
    for i in range(0, num_examples, batch_size):
        batch_indices = torch.tensor(
            indices[i: min(i + batch_size, num_examples)])
        yield features[batch_indices], labels[batch_indices]
```

In general, note that we want to use reasonably sized minibatches to take advantage of the GPU hardware, which excels at parallelizing operations. Because each example can be fed through our models in parallel and the gradient of the loss function for each example can also be taken in

parallel, GPUs allow us to process hundreds of examples in scarcely more time than it might take to process just a single example.

To build some intuition, let us read and print the first small batch of data examples. The shape of the features in each minibatch tells us both the minibatch size and the number of input features. Likewise, our minibatch of labels will have a shape given by `batch_size`.

```
batch_size = 10

for X, y in data_iter(batch_size, features, labels):
    print(X, '\n', y)
    break
```

```
tensor([[ -1.0999,  0.4855],
        [ 1.9538,  1.0919],
        [ 1.5107, -0.2886],
        [ 0.9734, -0.2309],
        [ 0.6716, -1.0258],
        [-0.9730, -0.7290],
        [-0.6671, -0.8925],
        [-0.7708, -0.3410],
        [ 0.0294, -1.2729],
        [-0.7009,  0.5991]])
tensor([[0.3487],
        [4.4049],
        [8.1914],
        [6.9287],
        [9.0329],
        [4.7465],
        [5.9153],
        [3.8213],
        [8.5887],
        [0.7470]])
```

As we run the iteration, we obtain distinct minibatches successively until the entire dataset has been exhausted (try this). While the iteration implemented above is good for didactic purposes, it is inefficient in ways that might get us in trouble on real problems. For example, it requires that we load all the data in memory and that we perform lots of random memory access. The built-in iterators implemented in a deep learning framework are considerably more efficient and they can deal with both data stored in files and data fed via data streams.

3.2.3 Initializing Model Parameters

Before we can begin optimizing our model's parameters by minibatch stochastic gradient descent, we need to have some parameters in the first place. In the following code, we initialize weights by sampling random numbers from a normal distribution with mean 0 and a standard deviation of 0.01, and setting the bias to 0.

```
w = torch.normal(0, 0.01, size=(2,1), requires_grad=True)
b = torch.zeros(1, requires_grad=True)
```

After initializing our parameters, our next task is to update them until they fit our data sufficiently well. Each update requires taking the gradient of our loss function with respect to the parameters.

Given this gradient, we can update each parameter in the direction that may reduce the loss.

Since nobody wants to compute gradients explicitly (this is tedious and error prone), we use automatic differentiation, as introduced in [Section 2.5](#), to compute the gradient.

3.2.4 Defining the Model

Next, we must define our model, relating its inputs and parameters to its outputs. Recall that to calculate the output of the linear model, we simply take the matrix-vector dot product of the input features $\mathbf{X}$ and the model weights $\mathbf{w}$, and add the offset b to each example. Note that below $\mathbf{X}\mathbf{w}$ is a vector and b is a scalar. Recall the broadcasting mechanism as described in [Section 2.1.3](#). When we add a vector and a scalar, the scalar is added to each component of the vector.

```
def linreg(X, w, b): #@save
    """The linear regression model."""
    return torch.matmul(X, w) + b
```

3.2.5 Defining the Loss Function

Since updating our model requires taking the gradient of our loss function, we ought to define the loss function first. Here we will use the squared loss function as described in [Section 3.1](#). In the implementation, we need to transform the true value y into the predicted value's shape y_{hat} . The result returned by the following function will also have the same shape as y_{hat} .

```
def squared_loss(y_hat, y): #@save
    """Squared loss."""
    return (y_hat - y.reshape(y_hat.shape)) ** 2 / 2
```

3.2.6 Defining the Optimization Algorithm

As we discussed in [Section 3.1](#), linear regression has a closed-form solution. However, this is not a book about linear regression: it is a book about deep learning. Since none of the other models that this book introduces can be solved analytically, we will take this opportunity to introduce your first working example of minibatch stochastic gradient descent.

At each step, using one minibatch randomly drawn from our dataset, we will estimate the gradient of the loss with respect to our parameters. Next, we will update our parameters in the direction that may reduce the loss. The following code applies the minibatch stochastic gradient descent update, given a set of parameters, a learning rate, and a batch size. The size of the update step is determined by the learning rate lr . Because our loss is calculated as a sum over the minibatch of examples, we normalize our step size by the batch size (`batch_size`), so that the magnitude of a typical step size does not depend heavily on our choice of the batch size.

```
def sgd(params, lr, batch_size): #@save
    """Minibatch stochastic gradient descent."""
    with torch.no_grad():
        for param in params:
            param -= lr * param.grad / batch_size
            param.grad.zero_()
```

3.2.7 Training

Now that we have all of the parts in place, we are ready to implement the main training loop. It is crucial that you understand this code because you will see nearly identical training loops over and over again throughout your career in deep learning.

In each iteration, we will grab a minibatch of training examples, and pass them through our model to obtain a set of predictions. After calculating the loss, we initiate the backwards pass through the network, storing the gradients with respect to each parameter. Finally, we will call the optimization algorithm `sgd` to update the model parameters.

In summary, we will execute the following loop:

- Initialize parameters $(\mathbf{w}, b)$
- Repeat until done
 - Compute gradient $\mathbf{g} \leftarrow \partial_{(\mathbf{w}, b)} \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} l(\mathbf{x}^{(i)}, y^{(i)}, \mathbf{w}, b)$
 - Update parameters $(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \eta \mathbf{g}$

In each *epoch*, we will iterate through the entire dataset (using the `data_iter` function) once passing through every example in the training dataset (assuming that the number of examples is divisible by the batch size). The number of epochs `num_epochs` and the learning rate `lr` are both hyperparameters, which we set here to 3 and 0.03, respectively. Unfortunately, setting hyperparameters is tricky and requires some adjustment by trial and error. We elide these details for now but revise them later in [Chapter 11](#).

```
lr = 0.03
num_epochs = 3
net = linreg
loss = squared_loss
```

```
for epoch in range(num_epochs):
    for X, y in data_iter(batch_size, features, labels):
        l = loss(net(X, w, b), y) # Minibatch loss in 'X' and 'y'
        # Compute gradient on 'l' with respect to ['w', 'b']
        l.sum().backward()
        sgd([w, b], lr, batch_size) # Update parameters using their gradient
    with torch.no_grad():
        train_l = loss(net(features, w, b), labels)
        print(f'epoch {epoch + 1}, loss {float(train_l.mean()):f}')
```

```
epoch 1, loss 0.035526
epoch 2, loss 0.000131
epoch 3, loss 0.000050
```

In this case, because we synthesized the dataset ourselves, we know precisely what the true parameters are. Thus, we can evaluate our success in training by comparing the true parameters with those that we learned through our training loop. Indeed they turn out to be very close to each other.

```
print(f'error in estimating w: {true_w - w.reshape(true_w.shape)}')
print(f'error in estimating b: {true_b - b}')
```

```
error in estimating w: tensor([1.6928e-05, 1.8287e-04], grad_fn=<SubBackward0>)
error in estimating b: tensor([0.0004], grad_fn=<RsubBackward1>)
```

Note that we should not take it for granted that we are able to recover the parameters perfectly. However, in machine learning, we are typically less concerned with recovering true underlying parameters, and more concerned with parameters that lead to highly accurate prediction. Fortunately, even on difficult optimization problems, stochastic gradient descent can often find remarkably good solutions, owing partly to the fact that, for deep networks, there exist many configurations of the parameters that lead to highly accurate prediction.

Summary

- We saw how a deep network can be implemented and optimized from scratch, using just tensors and auto differentiation, without any need for defining layers or fancy optimizers.
- This section only scratches the surface of what is possible. In the following sections, we will describe additional models based on the concepts that we have just introduced and learn how to implement them more concisely.

Exercises

1. What would happen if we were to initialize the weights to zero. Would the algorithm still work?
2. Assume that you are [Georg Simon Ohm](https://en.wikipedia.org/wiki/Georg_Ohm)⁴⁹ trying to come up with a model between voltage and current. Can you use auto differentiation to learn the parameters of your model?
3. Can you use [Planck's Law](https://en.wikipedia.org/wiki/Planck%27s_law)⁵⁰ to determine the temperature of an object using spectral energy density?
4. What are the problems you might encounter if you wanted to compute the second derivatives? How would you fix them?
5. Why is the reshape function needed in the squared_loss function?
6. Experiment using different learning rates to find out how fast the loss function value drops.
7. If the number of examples cannot be divided by the batch size, what happens to the data_iter function's behavior?

Discussions⁵¹

⁴⁹ https://en.wikipedia.org/wiki/Georg_Ohm

⁵⁰ https://en.wikipedia.org/wiki/Planck%27s_law

⁵¹ <https://discuss.d2l.ai/t/43>

3.3 Concise Implementation of Linear Regression

Broad and intense interest in deep learning for the past several years has inspired companies, academics, and hobbyists to develop a variety of mature open source frameworks for automating the repetitive work of implementing gradient-based learning algorithms. In [Section 3.2](#), we relied only on (i) tensors for data storage and linear algebra; and (ii) auto differentiation for calculating gradients. In practice, because data iterators, loss functions, optimizers, and neural network layers are so common, modern libraries implement these components for us as well.

In this section, we will show you how to implement the linear regression model from [Section 3.2](#) concisely by using high-level APIs of deep learning frameworks.

3.3.1 Generating the Dataset

To start, we will generate the same dataset as in [Section 3.2](#).

```
import numpy as np
import torch
from torch.utils import data
from d2l import torch as d2l
```

```
true_w = torch.tensor([2, -3.4])
true_b = 4.2
features, labels = d2l.synthetic_data(true_w, true_b, 1000)
```

3.3.2 Reading the Dataset

Rather than rolling our own iterator, we can call upon the existing API in a framework to read data. We pass in features and labels as arguments and specify `batch_size` when instantiating a data iterator object. Besides, the boolean value `is_train` indicates whether or not we want the data iterator object to shuffle the data on each epoch (pass through the dataset).

```
def load_array(data_arrays, batch_size, is_train=True): #@save
    """Construct a PyTorch data iterator."""
    dataset = data.TensorDataset(*data_arrays)
    return data.DataLoader(dataset, batch_size, shuffle=is_train)
```

```
batch_size = 10
data_iter = load_array((features, labels), batch_size)
```

Now we can use `data_iter` in much the same way as we called the `data_iter` function in [Section 3.2](#). To verify that it is working, we can read and print the first minibatch of examples. Comparing with [Section 3.2](#), here we use `iter` to construct a Python iterator and use `next` to obtain the first item from the iterator.

```
next(iter(data_iter))
```

```
[tensor([[ -0.0385, -1.5909],
         [ -0.1432,  0.9797],
         [-1.2551,  0.1154],
         [ 2.0345,  0.3657],
         [ 0.0179,  0.4323],
         [-1.9013, -0.6234],
         [-1.7319,  0.3589],
         [-0.2995,  0.0825],
         [-1.7680, -1.3965],
         [-0.8613, -0.5467]]),
 tensor([[ 9.5418],
         [ 0.5854],
         [ 1.3041],
         [ 7.0231],
         [ 2.7599],
         [ 2.5210],
         [-0.4856],
         [ 3.3295],
         [ 5.4176],
         [ 4.3249]])]
```

3.3.3 Defining the Model

When we implemented linear regression from scratch in [Section 3.2](#), we defined our model parameters explicitly and coded up the calculations to produce output using basic linear algebra operations. You *should* know how to do this. But once your models get more complex, and once you have to do this nearly every day, you will be glad for the assistance. The situation is similar to coding up your own blog from scratch. Doing it once or twice is rewarding and instructive, but you would be a lousy web developer if every time you needed a blog you spent a month reinventing the wheel.

For standard operations, we can use a framework’s predefined layers, which allow us to focus especially on the layers used to construct the model rather than having to focus on the implementation. We will first define a model variable `net`, which will refer to an instance of the `Sequential` class. The `Sequential` class defines a container for several layers that will be chained together. Given input data, a `Sequential` instance passes it through the first layer, in turn passing the output as the second layer’s input and so forth. In the following example, our model consists of only one layer, so we do not really need `Sequential`. But since nearly all of our future models will involve multiple layers, we will use it anyway just to familiarize you with the most standard workflow.

Recall the architecture of a single-layer network as shown in [Fig. 3.1.2](#). The layer is said to be *fully-connected* because each of its inputs is connected to each of its outputs by means of a matrix-vector multiplication.

In PyTorch, the fully-connected layer is defined in the `Linear` class. Note that we passed two arguments into `nn.Linear`. The first one specifies the input feature dimension, which is 2, and the second one is the output feature dimension, which is a single scalar and therefore 1.

```
# `nn` is an abbreviation for neural networks
from torch import nn

net = nn.Sequential(nn.Linear(2, 1))
```

3.3.4 Initializing Model Parameters

Before using `net`, we need to initialize the model parameters, such as the weights and bias in the linear regression model. Deep learning frameworks often have a predefined way to initialize the parameters. Here we specify that each weight parameter should be randomly sampled from a normal distribution with mean 0 and standard deviation 0.01. The bias parameter will be initialized to zero.

As we have specified the input and output dimensions when constructing `nn.Linear`, now we can access the parameters directly to specify their initial values. We first locate the layer by `net[0]`, which is the first layer in the network, and then use the `weight.data` and `bias.data` methods to access the parameters. Next we use the replace methods `normal_` and `fill_` to overwrite parameter values.

```
net[0].weight.data.normal_(0, 0.01)
net[0].bias.data.fill_(0)
```

```
tensor([0.])
```

3.3.5 Defining the Loss Function

The `MSELoss` class computes the mean squared error (without the $1/2$ factor in (3.1.5)). By default it returns the average loss over examples.

```
loss = nn.MSELoss()
```

3.3.6 Defining the Optimization Algorithm

Minibatch stochastic gradient descent is a standard tool for optimizing neural networks and thus PyTorch supports it alongside a number of variations on this algorithm in the `optim` module. When we instantiate an SGD instance, we will specify the parameters to optimize over (obtainable from our `net` via `net.parameters()`), with a dictionary of hyperparameters required by our optimization algorithm. Minibatch stochastic gradient descent just requires that we set the value `lr`, which is set to 0.03 here.

```
trainer = torch.optim.SGD(net.parameters(), lr=0.03)
```

3.3.7 Training

You might have noticed that expressing our model through high-level APIs of a deep learning framework requires comparatively few lines of code. We did not have to individually allocate parameters, define our loss function, or implement minibatch stochastic gradient descent. Once we start working with much more complex models, advantages of high-level APIs will grow considerably. However, once we have all the basic pieces in place, the training loop itself is strikingly similar to what we did when implementing everything from scratch.

To refresh your memory: for some number of epochs, we will make a complete pass over the dataset (`train_data`), iteratively grabbing one minibatch of inputs and the corresponding ground-truth labels. For each minibatch, we go through the following ritual:

- Generate predictions by calling `net(X)` and calculate the loss `l` (the forward propagation).
- Calculate gradients by running the backpropagation.
- Update the model parameters by invoking our optimizer.

For good measure, we compute the loss after each epoch and print it to monitor progress.

```
num_epochs = 3
for epoch in range(num_epochs):
    for X, y in data_iter:
        l = loss(net(X), y)
        trainer.zero_grad()
        l.backward()
        trainer.step()
    l = loss(net(features), labels)
    print(f'epoch {epoch + 1}, loss {l:f}')
```

```
epoch 1, loss 0.000207
epoch 2, loss 0.000105
epoch 3, loss 0.000105
```

Below, we compare the model parameters learned by training on finite data and the actual parameters that generated our dataset. To access parameters, we first access the layer that we need from `net` and then access that layer's weights and bias. As in our from-scratch implementation, note that our estimated parameters are close to their ground-truth counterparts.

```
w = net[0].weight.data
print('error in estimating w:', true_w - w.reshape(true_w.shape))
b = net[0].bias.data
print('error in estimating b:', true_b - b)
```

```
error in estimating w: tensor([-0.0005, -0.0003])
error in estimating b: tensor([-0.0002])
```

Summary

- Using PyTorch's high-level APIs, we can implement models much more concisely.
- In PyTorch, the `data` module provides tools for data processing, the `nn` module defines a large number of neural network layers and common loss functions.
- We can initialize the parameters by replacing their values with methods ending with `_`.

Exercises

1. If we replace `nn.MSELoss(reduction='sum')` with `nn.MSELoss()`, how can we change the learning rate for the code to behave identically. Why?
2. Review the PyTorch documentation to see what loss functions and initialization methods are provided. Replace the loss by Huber's loss.
3. How do you access the gradient of `net[0].weight`?

Discussions⁵²

3.4 Softmax Regression

In Section 3.1, we introduced linear regression, working through implementations from scratch in Section 3.2 and again using high-level APIs of a deep learning framework in Section 3.3 to do the heavy lifting.

Regression is the hammer we reach for when we want to answer *how much?* or *how many?* questions. If you want to predict the number of dollars (price) at which a house will be sold, or the number of wins a baseball team might have, or the number of days that a patient will remain hospitalized before being discharged, then you are probably looking for a regression model.

In practice, we are more often interested in *classification*: asking not “how much” but “which one”:

- Does this email belong in the spam folder or the inbox?
- Is this customer more likely *to sign up* or *not to sign up* for a subscription service?
- Does this image depict a donkey, a dog, a cat, or a rooster?
- Which movie is Aston most likely to watch next?

Colloquially, machine learning practitioners overload the word *classification* to describe two subtly different problems: (i) those where we are interested only in hard assignments of examples to categories (classes); and (ii) those where we wish to make soft assignments, i.e., to assess the probability that each category applies. The distinction tends to get blurred, in part, because often, even when we only care about hard assignments, we still use models that make soft assignments.

3.4.1 Classification Problem

To get our feet wet, let us start off with a simple image classification problem. Here, each input consists of a 2×2 grayscale image. We can represent each pixel value with a single scalar, giving us four features x_1, x_2, x_3, x_4 . Further, let us assume that each image belongs to one among the categories “cat”, “chicken”, and “dog”.

Next, we have to choose how to represent the labels. We have two obvious choices. Perhaps the most natural impulse would be to choose $y \in \{1, 2, 3\}$, where the integers represent {dog, cat, chicken} respectively. This is a great way of *storing* such information on a computer. If the categories had some natural ordering among them, say if we were trying to predict {baby, toddler, adolescent, young adult, adult, geriatric}, then it might even make sense to cast this problem as regression and keep the labels in this format.

⁵² <https://discuss.d2l.ai/t/45>

But general classification problems do not come with natural orderings among the classes. Fortunately, statisticians long ago invented a simple way to represent categorical data: the *one-hot encoding*. A one-hot encoding is a vector with as many components as we have categories. The component corresponding to particular instance's category is set to 1 and all other components are set to 0. In our case, a label y would be a three-dimensional vector, with $(1, 0, 0)$ corresponding to “cat”, $(0, 1, 0)$ to “chicken”, and $(0, 0, 1)$ to “dog”:

$$y \in \{(1, 0, 0), (0, 1, 0), (0, 0, 1)\}. \quad (3.4.1)$$

3.4.2 Network Architecture

In order to estimate the conditional probabilities associated with all the possible classes, we need a model with multiple outputs, one per class. To address classification with linear models, we will need as many affine functions as we have outputs. Each output will correspond to its own affine function. In our case, since we have 4 features and 3 possible output categories, we will need 12 scalars to represent the weights (w with subscripts), and 3 scalars to represent the biases (b with subscripts). We compute these three *logits*, o_1, o_2 , and o_3 , for each input:

$$\begin{aligned} o_1 &= x_1w_{11} + x_2w_{12} + x_3w_{13} + x_4w_{14} + b_1, \\ o_2 &= x_1w_{21} + x_2w_{22} + x_3w_{23} + x_4w_{24} + b_2, \\ o_3 &= x_1w_{31} + x_2w_{32} + x_3w_{33} + x_4w_{34} + b_3. \end{aligned} \quad (3.4.2)$$

We can depict this calculation with the neural network diagram shown in Fig. 3.4.1. Just as in linear regression, softmax regression is also a single-layer neural network. And since the calculation of each output, o_1, o_2 , and o_3 , depends on all inputs, x_1, x_2, x_3 , and x_4 , the output layer of softmax regression can also be described as fully-connected layer.

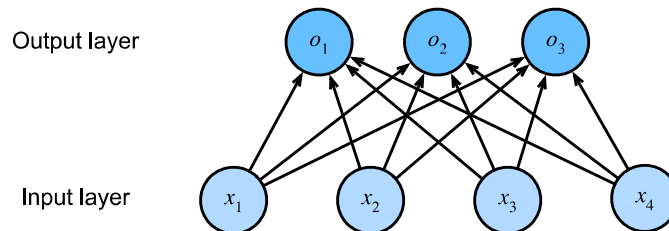


Fig. 3.4.1: Softmax regression is a single-layer neural network.

To express the model more compactly, we can use linear algebra notation. In vector form, we arrive at $\mathbf{o} = \mathbf{W}\mathbf{x} + \mathbf{b}$, a form better suited both for mathematics, and for writing code. Note that we have gathered all of our weights into a 3×4 matrix and that for features of a given data example $\mathbf{x}$, our outputs are given by a matrix-vector product of our weights by our input features plus our biases $\mathbf{b}$.

3.4.3 Parameterization Cost of Fully-Connected Layers

As we will see in subsequent chapters, fully-connected layers are ubiquitous in deep learning. However, as the name suggests, fully-connected layers are *fully* connected with potentially many learnable parameters. Specifically, for any fully-connected layer with d inputs and q outputs, the parameterization cost is $\mathcal{O}(dq)$, which can be prohibitively high in practice. Fortunately, this cost of transforming d inputs into q outputs can be reduced to $\mathcal{O}(\frac{dq}{n})$, where the hyperparameter n can be flexibly specified by us to balance between parameter saving and model effectiveness in real-world applications (Zhang et al., 2021).

3.4.4 Softmax Operation

The main approach that we are going to take here is to interpret the outputs of our model as probabilities. We will optimize our parameters to produce probabilities that maximize the likelihood of the observed data. Then, to generate predictions, we will set a threshold, for example, choosing the label with the maximum predicted probabilities.

Put formally, we would like any output $\hat{y}_j$ to be interpreted as the probability that a given item belongs to class j . Then we can choose the class with the largest output value as our prediction $\operatorname{argmax}_j y_j$. For example, if $\hat{y}_1$, $\hat{y}_2$, and $\hat{y}_3$ are 0.1, 0.8, and 0.1, respectively, then we predict category 2, which (in our example) represents “chicken”.

You might be tempted to suggest that we interpret the logits $\mathbf{o}$ directly as our outputs of interest. However, there are some problems with directly interpreting the output of the linear layer as a probability. On one hand, nothing constrains these numbers to sum to 1. On the other hand, depending on the inputs, they can take negative values. These violate basic axioms of probability presented in Section 2.6

To interpret our outputs as probabilities, we must guarantee that (even on new data), they will be nonnegative and sum up to 1. Moreover, we need a training objective that encourages the model to estimate faithfully probabilities. Of all instances when a classifier outputs 0.5, we hope that half of those examples will actually belong to the predicted class. This is a property called *calibration*.

The *softmax function*, invented in 1959 by the social scientist R. Duncan Luce in the context of *choice models*, does precisely this. To transform our logits such that they become nonnegative and sum to 1, while requiring that the model remains differentiable, we first exponentiate each logit (ensuring non-negativity) and then divide by their sum (ensuring that they sum to 1):

$$\hat{\mathbf{y}} = \operatorname{softmax}(\mathbf{o}) \quad \text{where} \quad \hat{y}_j = \frac{\exp(o_j)}{\sum_k \exp(o_k)}. \quad (3.4.3)$$

It is easy to see $\hat{y}_1 + \hat{y}_2 + \hat{y}_3 = 1$ with $0 \leq \hat{y}_j \leq 1$ for all j . Thus, $\hat{\mathbf{y}}$ is a proper probability distribution whose element values can be interpreted accordingly. Note that the softmax operation does not change the ordering among the logits $\mathbf{o}$, which are simply the pre-softmax values that determine the probabilities assigned to each class. Therefore, during prediction we can still pick out the most likely class by

$$\operatorname{argmax}_j \hat{y}_j = \operatorname{argmax}_j o_j. \quad (3.4.4)$$

Although softmax is a nonlinear function, the outputs of softmax regression are still *determined* by an affine transformation of input features; thus, softmax regression is a linear model.

3.4.5 Vectorization for Minibatches

To improve computational efficiency and take advantage of GPUs, we typically carry out vector calculations for minibatches of data. Assume that we are given a minibatch $\mathbf{X}$ of examples with feature dimensionality (number of inputs) d and batch size n . Moreover, assume that we have q categories in the output. Then the minibatch features $\mathbf{X}$ are in $\mathbb{R}^{n \times d}$, weights $\mathbf{W} \in \mathbb{R}^{d \times q}$, and the bias satisfies $\mathbf{b} \in \mathbb{R}^{1 \times q}$.

$$\begin{aligned}\mathbf{O} &= \mathbf{XW} + \mathbf{b}, \\ \hat{\mathbf{Y}} &= \text{softmax}(\mathbf{O}).\end{aligned}\tag{3.4.5}$$

This accelerates the dominant operation into a matrix-matrix product $\mathbf{XW}$ vs. the matrix-vector products we would be executing if we processed one example at a time. Since each row in $\mathbf{X}$ represents a data example, the softmax operation itself can be computed *rowwise*: for each row of $\mathbf{O}$, exponentiate all entries and then normalize them by the sum. Triggering broadcasting during the summation $\mathbf{XW} + \mathbf{b}$ in (3.4.5), both the minibatch logits $\mathbf{O}$ and output probabilities $\hat{\mathbf{Y}}$ are $n \times q$ matrices.

3.4.6 Loss Function

Next, we need a loss function to measure the quality of our predicted probabilities. We will rely on maximum likelihood estimation, the very same concept that we encountered when providing a probabilistic justification for the mean squared error objective in linear regression (Section 3.1.3).

Log-Likelihood

The softmax function gives us a vector $\hat{\mathbf{y}}$, which we can interpret as estimated conditional probabilities of each class given any input $\mathbf{x}$, e.g., $\hat{y}_1 = P(y = \text{cat} \mid \mathbf{x})$. Suppose that the entire dataset $\{\mathbf{X}, \mathbf{Y}\}$ has n examples, where the example indexed by i consists of a feature vector $\mathbf{x}^{(i)}$ and a one-hot label vector $\mathbf{y}^{(i)}$. We can compare the estimates with reality by checking how probable the actual classes are according to our model, given the features:

$$P(\mathbf{Y} \mid \mathbf{X}) = \prod_{i=1}^n P(\mathbf{y}^{(i)} \mid \mathbf{x}^{(i)}).\tag{3.4.6}$$

According to maximum likelihood estimation, we maximize $P(\mathbf{Y} \mid \mathbf{X})$, which is equivalent to minimizing the negative log-likelihood:

$$-\log P(\mathbf{Y} \mid \mathbf{X}) = \sum_{i=1}^n -\log P(\mathbf{y}^{(i)} \mid \mathbf{x}^{(i)}) = \sum_{i=1}^n l(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)}),\tag{3.4.7}$$

where for any pair of label $\mathbf{y}$ and model prediction $\hat{\mathbf{y}}$ over q classes, the loss function l is

$$l(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{j=1}^q y_j \log \hat{y}_j.\tag{3.4.8}$$

For reasons explained later on, the loss function in (3.4.8) is commonly called the *cross-entropy loss*. Since $\mathbf{y}$ is a one-hot vector of length q , the sum over all its coordinates j vanishes for all but one term. Since all $\hat{y}_j$ are predicted probabilities, their logarithm is never larger than 0. Consequently, the loss function cannot be minimized any further if we correctly predict the actual label with

certainty, i.e., if the predicted probability $P(\mathbf{y} \mid \mathbf{x}) = 1$ for the actual label $\mathbf{y}$. Note that this is often impossible. For example, there might be label noise in the dataset (some examples may be mislabeled). It may also not be possible when the input features are not sufficiently informative to classify every example perfectly.

Softmax and Derivatives

Since the softmax and the corresponding loss are so common, it is worth understanding a bit better how it is computed. Plugging (3.4.3) into the definition of the loss in (3.4.8) and using the definition of the softmax we obtain:

$$\begin{aligned} l(\mathbf{y}, \hat{\mathbf{y}}) &= - \sum_{j=1}^q y_j \log \frac{\exp(o_j)}{\sum_{k=1}^q \exp(o_k)} \\ &= \sum_{j=1}^q y_j \log \sum_{k=1}^q \exp(o_k) - \sum_{j=1}^q y_j o_j \\ &= \log \sum_{k=1}^q \exp(o_k) - \sum_{j=1}^q y_j o_j. \end{aligned} \tag{3.4.9}$$

To understand a bit better what is going on, consider the derivative with respect to any logit o_j . We get

$$\partial_{o_j} l(\mathbf{y}, \hat{\mathbf{y}}) = \frac{\exp(o_j)}{\sum_{k=1}^q \exp(o_k)} - y_j = \text{softmax}(\mathbf{o})_j - y_j. \tag{3.4.10}$$

In other words, the derivative is the difference between the probability assigned by our model, as expressed by the softmax operation, and what actually happened, as expressed by elements in the one-hot label vector. In this sense, it is very similar to what we saw in regression, where the gradient was the difference between the observation y and estimate $\hat{y}$. This is not coincidence. In any exponential family (see the [online appendix on distributions](#)⁵³) model, the gradients of the log-likelihood are given by precisely this term. This fact makes computing gradients easy in practice.

Cross-Entropy Loss

Now consider the case where we observe not just a single outcome but an entire distribution over outcomes. We can use the same representation as before for the label $\mathbf{y}$. The only difference is that rather than a vector containing only binary entries, say $(0, 0, 1)$, we now have a generic probability vector, say $(0.1, 0.2, 0.7)$. The math that we used previously to define the loss l in (3.4.8) still works out fine, just that the interpretation is slightly more general. It is the expected value of the loss for a distribution over labels. This loss is called the *cross-entropy loss* and it is one of the most commonly used losses for classification problems. We can demystify the name by introducing just the basics of information theory. If you wish to understand more details of information theory, you may further refer to the [online appendix on information theory](#)⁵⁴.

⁵³ https://d2l.ai/chapter_appendix-mathematics-for-deep-learning/distributions.html

⁵⁴ https://d2l.ai/chapter_appendix-mathematics-for-deep-learning/information-theory.html

3.4.7 Information Theory Basics

Information theory deals with the problem of encoding, decoding, transmitting, and manipulating information (also known as data) in as concise form as possible.

Entropy

The central idea in information theory is to quantify the information content in data. This quantity places a hard limit on our ability to compress the data. In information theory, this quantity is called the *entropy* of a distribution P , and it is captured by the following equation:

$$H[P] = \sum_j -P(j) \log P(j). \quad (3.4.11)$$

One of the fundamental theorems of information theory states that in order to encode data drawn randomly from the distribution P , we need at least $H[P]$ “nats” to encode it. If you wonder what a “nat” is, it is the equivalent of bit but when using a code with base e rather than one with base 2. Thus, one nat is $\frac{1}{\log(2)} \approx 1.44$ bit.

Surprisal

You might be wondering what compression has to do with prediction. Imagine that we have a stream of data that we want to compress. If it is always easy for us to predict the next token, then this data is easy to compress! Take the extreme example where every token in the stream always takes the same value. That is a very boring data stream! And not only it is boring, but it is also easy to predict. Because they are always the same, we do not have to transmit any information to communicate the contents of the stream. Easy to predict, easy to compress.

However if we cannot perfectly predict every event, then we might sometimes be surprised. Our surprise is greater when we assigned an event lower probability. Claude Shannon settled on $\log \frac{1}{P(j)} = -\log P(j)$ to quantify one’s *surprisal* at observing an event j having assigned it a (subjective) probability $P(j)$. The entropy defined in (3.4.11) is then the *expected surprisal* when one assigned the correct probabilities that truly match the data-generating process.

Cross-Entropy Revisited

So if entropy is level of surprise experienced by someone who knows the true probability, then you might be wondering, what is cross-entropy? The cross-entropy from P to Q , denoted $H(P, Q)$, is the expected surprisal of an observer with subjective probabilities Q upon seeing data that were actually generated according to probabilities P . The lowest possible cross-entropy is achieved when $P = Q$. In this case, the cross-entropy from P to Q is $H(P, P) = H(P)$.

In short, we can think of the cross-entropy classification objective in two ways: (i) as maximizing the likelihood of the observed data; and (ii) as minimizing our surprisal (and thus the number of bits) required to communicate the labels.